

Presentation – Datamodellering för YrkesCo

1. Introduktion

YrkesCo har ett behov av att samla data relaterat till utbildningsverksamhet på ett strukturerat, integrerat och säkert sätt.

Målet har därför varit att designa en relationsdatabas som kan stödja:

- Studenter
- Utbildare (både konsulter och anställda)
- Utbildningsledare
- Program och kurser
- Klasstilldelningar
- Företag kopplade till utbildare
- Platsdata (anläggningar och adresser)

Datamodellen har utvecklats enligt principerna för datamodellering och 3NF.

2. Övergripande datamodellering (Uppgift 0)

a) Konceptuell modell

Den konceptuella modellen beskriver entiteterna och deras relationer på en övergripande nivå utan teknisk detalj.

b) Relationship statements

Exempel på relationer mellan entiteter:

- Ett Program består av flera Kurs(er) (M:N via ProgramKurs)
- En Kurs kan undervisas av flera Utbildare (M:N via Kurs_Utbildare)
- En Utbildningsledare ansvarar för 3 klasser (1:M)
- En Student tillhör exakt en Klass (M:1)
- En Klass kopplas till ett Program
- En Utbildare kan vara kopplad till ett Företag (1:M)
- En Anläggning finns på en Adress (1:1)
- En Adress ligger i ett Land (M:1)

3. Logisk modell

Den logiska modellen omvandlar den konceptuella designen till en databasstruktur med:

Tabeller

Attribut

Primärnycklar (PK)

Främmande nycklar (FK)

Kardinaliteter

Nedan följer ett urval av centrala tabeller och deras primärnycklar:

Program(id)

Kurs(id)

Student(id)

Klass(id)

Utbildare(id)

Utbildningsledare(id)

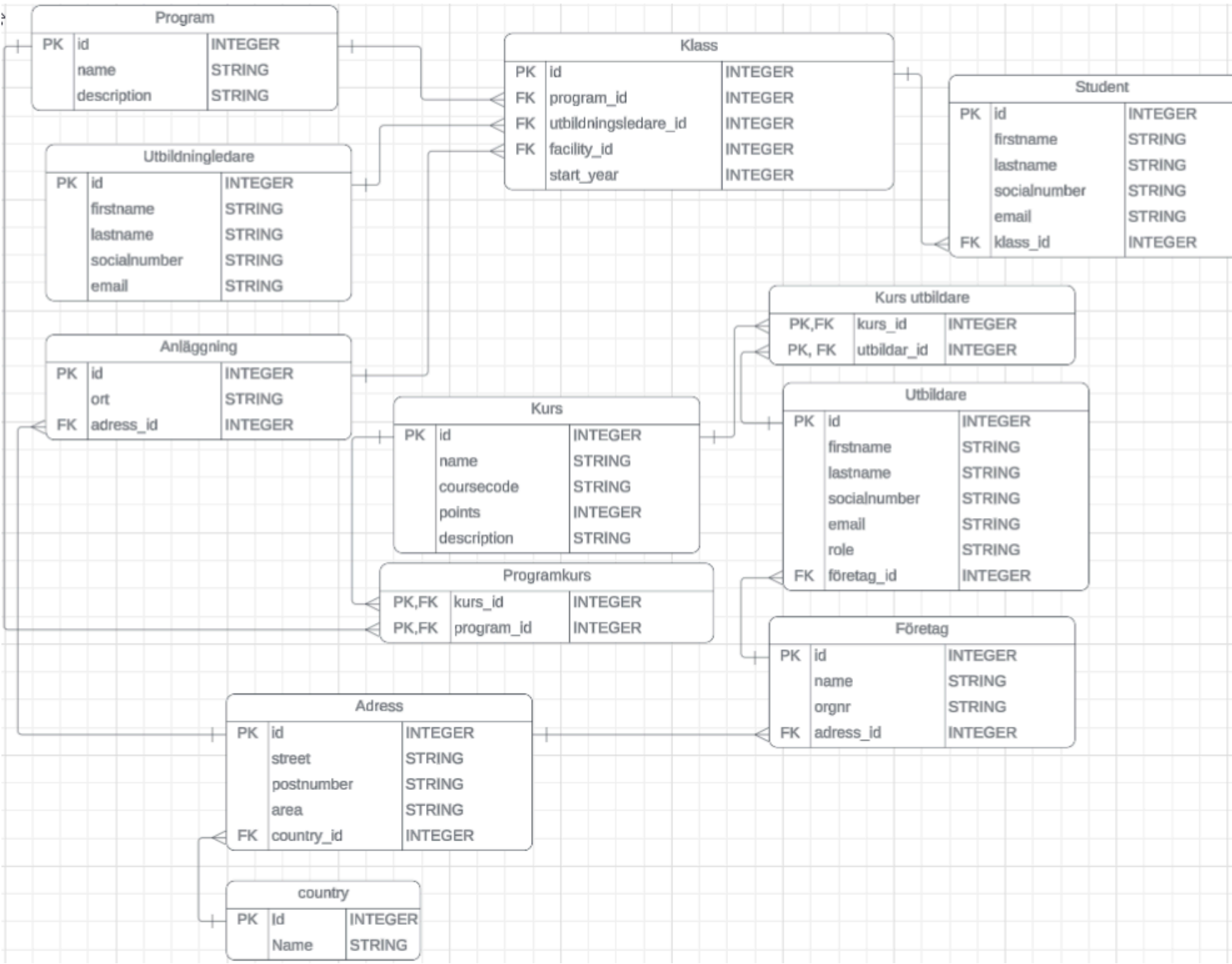
Företag(id)

Adress(id) → Country(id)

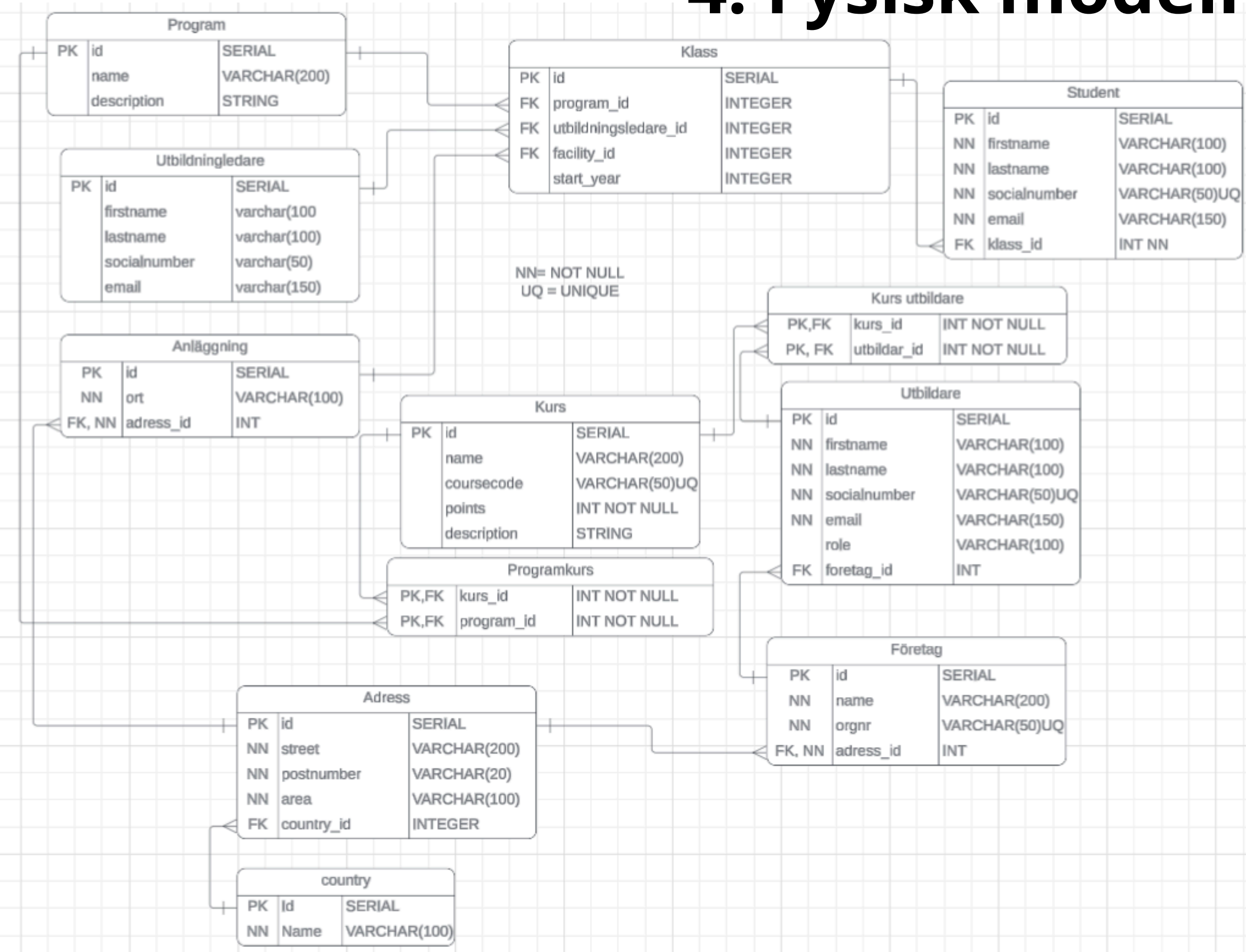
M:N-relationer löses via bryggtabeller:

ProgramKurs(program_id, kurs_id)

Kurs_Utbildare(kurs_id, utbildar_id)



4. Fysisk modell



Den fysiska modellen beskriver implementationen i PostgreSQL:

Exempel:

- SERIAL används för auto-increment PK
- VARCHAR definierar textfält
- FOREIGN KEY används för relationer
- UNIQUE används där data inte får dupliceras (t.ex. personnummer, coursecode)

Den fysiska modellen ligger i ett ddl.sql skript tillsammans med en dml.sql fil för seed-data samt en query.sql för queries.

5. Normalisering och 3NF

Datamodellen är konstruerad för att uppnå tredje normalformen (3NF).

Motivering:

1NF:

1. Alla tabeller har atomära värden och unika rader.

2NF:

2. Alla icke-nyckelattribut beror på hela nyckeln.
3. M:N-relationer bryts ut i egna tabeller.

3NF:

4. Inga transitiva beroenden finns. Exempel:
 - Adress har egen tabell → eliminerar upprepade fält
 - Land har egen tabell → undviker upprepade strängvärden
 - Företag och Utbildare separeras → undviker redundans

Resultatet är en databas som:

- ✓ minskar redundans
- ✓ förbättrar datakvalitet
- ✓ förenklar accesskontroll för känsliga uppgifter

6. Implementation i PostgreSQL (Uppgift 1)

Implementation består av tre delar:

a) Skapa tabeller

Alla tabeller skapas via docker-init.sql

b) Seed-data

Varje tabell fylls med minst 3 testposter för att möjliggöra joins och demos.

c) Test-queries

Exempel på körbar query:

```
-- 3NF relation
SELECT u.firstname || ' ' || u.lastname AS utbildare,
       f.name AS foretag,
       a.street,
       a.area,
       c.name AS land
FROM utbildare u
JOIN foretag f ON u.foretag_id = f.id
JOIN adress a ON f.adress_id = a.id
JOIN country c ON a.country_id = c.id;
```

```
-- specifik search for program name
SELECT p.name AS program,
       k.name AS kurs,
       k.coursecode,
       k.points
FROM program p
JOIN programkurs pk ON p.id = pk.program_id
JOIN kurs k ON pk.kurs_id = k.id
WHERE p.name = 'Systemutvecklare .NET';
```

utbildare	foretag	street	area	land
Anna Berg	Tech AB	Sveavägen 12	Stockholm	Sweden
Lars Nilsen	Nordic Solutions AS	Karl Johans gate 22	Oslo	Norway
Mia Korhonen	EduConsult Oy	Mannerheimintie 45	Helsinki	Finland
(3 rows)				
program	kurs	coursecode	points	
Systemutvecklare .NET	C# Backend Development	CSHARP01	40	
Systemutvecklare .NET	Databasteknik SQL	DBSQL01	30	
(2 rows)				

7. Sammanfattning

Datamodellen för YrkesCo är:

- ✓ Normaliserad till 3NF
- ✓ Implementerad i PostgreSQL
- ✓ Docker-baserad för repeterbar utvecklingsmiljö
- ✓ Bygger på tydliga M:N, 1:M och 1:1 relationer
- ✓ Skalbar för framtida funktioner som fristående kurser och flera anläggningar

Databasen kommer att möjliggöra bättre informationsflöde för:

- Administratörer
- Utbildningsledare
- Ekonomi/HR
- Drift/IT

Samt skapa mindre redundans, bättre kvalitet och högre dataintegritet.